

PROGRAMOWANIE NISKOPOZIOMOWE

NOTATKI

„Mam nadzieję, że nie będzie na niskim poziomie.”
„To było na systemach operacyjnych.”

MACIEJ MIKOŁAJCZAK
Kraków, 2025

Spis treści

1. Architektura x86	2
---------------------	---

1. Architektura x86

Procesor posiada pamięć wewnętrzną – rejestry. Są to pojedyncze komórki pamięci, są potrzebne, bo trzeba mieć jakąś minimalną pamięć potrzebną do komunikacji z RAM'em.

Pierwotnie były rejestry **ax**, **bx**, **cx**, **dx** i rejestry specjalne **si**, **di**, **sp**, **bp** – tak było w architekturze 16-bitowej.

Potem zmieniono na 32-bitowe – dodajemy do nazwy na początek **e** jak extended.

Na 64-bitach zamiast **e** jest **r** i dodano 8 nowych rejestrów – **r8**, **r9**, ..., **r15**.

Ostatecznie jest **rax**, **rbx**, **rcx**, **rdx**, **rsi**, **rbi**, **rsp**, **rbp**, **r8**, ..., **r15**.

Stare nazwy dalej obowiązują – 32 dolne bity **rax** funkcjonuje pod nazwą **eax**, a dolne 16 – **ax**, do tego jest jeszcze **ah**, **al** – górna i dolna połowa **ax**, tak było od początku.

W rejestrach specjalnych wygląda to tak samo, nie ma podziału na części 8-bitowe, jest tylko dolna – **si** -> **sil**, nawet ona nie jest dostępna zawsze, tylko w niektórych instrukcjach.

W najnowszych rejestrach mamy nową nomenklaturę:

- **b** – jeden bajt
- **w** – słowo, dwa bajty (bo kiedyś była architektura 16-bitowa)
- **d** – double word, cztery bajty, bo to podwójne słowo gdy rozumiemy słowo jako 16 bajtów
- **q** – quad word, osiem bajtów

r9 -> **r9d**, **r9w**, **r9b** dają nam dostęp do dolnych części rejestru o danej długości.

Tryb pracy procesora może oznaczać ilość bitów używanych do adresowania pamięci lub ilość bitów rejestru. Standardowo myślimy o tym pierwszym. W procesorze 16-bitowym mamy maksymalnie 2^{16} adresów, to jest dość mało, dlatego wprowadzono mechanizm segmentacji – mamy 16 bitów segmentu i 16 offsetu, to znacząco zwiększa ilość dostępnej pamięci. Jako, że segment rzadko się zmienia, to można ustalić segment i dalej pracować na wskaźniku 16-bitowym, segment zmienić jak będzie to potrzebne. Dlatego będą do tego osobne instrukcje.

Początkowo pamięć nie była chroniona – operowano na adresach fizycznych, więc każdy mógł zmienić pamięć. Wzmocniono segmentację – system operacyjny ustala, gdzie programy widzą swoje segmenty i upewnia się, że nikt nie odwołuje się poza swoją pamięć. To się stało już, gdy były procesory 32-bitowe.

Drugi mechanizm – adresy wirtualne, adres ma normalnie 32 bity, dzielimy je na 12 dolnych i resztę – ta reszta oznacza stronę pamięci, która ma 2^{12} bitów. System operacyjny ustala to, do czego w pamięci fizycznej odnosi się strona. Dzięki temu mamy możliwość chronienia pamięci, jak i jej współdzielenia.

W ten sposób otrzymujemy następujące tryby pracy procesora:

- Legacy Mode:
 - Real Mode – 16 bitów, prosta segmentacja, nie chroniona pamięć, 1MB dostępnej pamięci.
 - Protected Mode – 32 bity, segmentacja, stronicowanie, chroniona pamięć, 4GB pamięci.
 - Virtual-8086 – uruchamianie 16-bitowych programów w trybie Protected.
- Long Mode:
 - 64-Bit Mode – brak segmentacji (bo już możemy adresować dużo pamięci), za to stronicowanie, chroniona pamięć, rejestry i adresowanie 64-bitowe.
 - Compatibility Mode – dla użytkownika działa jak Protected Mode, dla systemu jak 64-Bit.

W Legacy i Compatibility Mode mamy do dyspozycji kilka różnych segmentów, w 64-Bit Mode zostają nam tylko trzy – CS odpowiedzialny za przechowywanie atrybutów (np. uprawnień), FS i GS używane do robienia rzeczy thread-local.

W instrukcjach będziemy używać różnych sposobów adresowania:

- Absolute – pełny adres podawany w instrukcji.
- ModR/M:

$$\text{address} = \text{base} + \text{index} \cdot \text{scale} + \text{displacement},$$

gdzie *base* i *index* są w rejestrach, $\text{scale} \in \{0, 1, 2, 4, 8\}$ i *displacement* są podane w instrukcji. To pozwala nam na ładne przechodzenie po pamięci.

- Instruction-relative:

$$\text{address} = \text{RIP} + \text{displacement},$$

czyli przesuwamy się od obecnej instrukcji, to pozwala umieszczać dane wspólnie z kodem.

- Implicit – adres jest wyznaczony przez typ instrukcji.

W Assemblerze nie mamy typów zmiennych, tylko ciągi bitów. Ich interpretacja jest zawarta w instrukcji. Instrukcja identyfikuje rozmiar (byte, word, doubleword, quadword, double quadword), położenie (stała podana w instrukcji, rejestr, adres w pamięci, port I/O) i interpretację (signed int, unsigned int, BCD – Binary Coded Decimal, Packed BCD digits, strings, floats).

BCD trzyma każdą cyfrę w kolejnym bajcie, ale właściwie są do tego potrzebne 4 bity, dlatego Packed BCD trzyma po dwie cyfry w bajcie.

Pracujemy na wartościach wielobajtowych, można te bajty pisać od najmniej lub najbardziej znaczących. Jeśli wpisujemy „po ludzku” – od lewej do prawej, to najbardziej znaczący bajt ma najmniejszy adres. Dlatego drugi tryb też ma sens.

Little endian – pod najniższym adresem jest najniższy bajt, tak działa x86 i amd64.

Big endian – pod najniższym adresem jest najwyższy bajt, np. PlayStation i Xbox, protokoły sieciowe.

Wielka lista instrukcji:

- Transfer danych – `mov`, `cmovXX`, `push`, `pop`.
- Arytmetyczne – `add`, `sub`, `neg`, `mul`, `imul` (mnożenie naturalne i całkowite), `div`, `idiv`, `dec`, `inc`, `lea`.
- Logiczne i bitowe – `and`, `or`, `xor`, `not`, `shl`, `shr`.
- Testowanie i porównania – `cmp`, `test`, `bt`, `setXX`.
- Sterowanie przepływem – `jmp`, `jXX`, `loop`, `call`, `ret`.
- Wywołania systemowe – `syscall`, `sysret`, `sysenter`, `sysexit`.

Rejestr znaczników to rejestr, którego pojedyncze bity mają pewną interpretację. Niektóre np. mówią nam coś o wyniku ostatniej instrukcji. Jest np. carry flag – wynik wyszedł poza liczbę, nastąpiło przeniesienie, parity flag – w wyniku jest parzysta liczba jedynek.

Instrukcja `jXX` oznacza skok warunkowy, gdzie warunek zależy od któregoś ze znaczników. Podczas instrukcji `sub A B` ustawiane są znaczniki, z których można np. wyciągnąć, która liczba jest większa (ważne jest to, że ustawią się różnie zależnie od tego, czy liczby mają znaki).